

FleetX

Vehicle Rental & Fleet Analytics Platform

The Complete Handbook

Full-Stack Development (React + Spring Boot) | Security (JWT) | Data Analytics (SQL + Python)

Written for someone starting from zero knowledge of this project.

0 How to Use This Handbook / Yeh Handbook Kaise Padhein

Yeh handbook is tarah banayi gayi hai ki agar tumne pehle kabhi FleetX project ke baare mein kuch nahi suna, tab bhi tum ise shuru se aakhir tak padh ke poora project samajh jaoge — jaise ki tumne khud banaya ho.

Har concept ko simple bhasha mein samjhaya gaya hai. Jahan zaroori laga, technical English term ke saath Hinglish explanation bhi di gayi hai. Isse ek interview preparation guide ki tarah bhi use kar sakte ho — end mein common interview questions aur unke answers bhi diye gaye hain.

Handbook mein kya-kya hai

- Chapter 1: Project kya hai aur kyun banaya
- Chapter 2: Tech stack — har technology ka matlab
- Chapter 3: Poora architecture — sab kuch kaise connect hai
- Chapter 4: Database design — saari tables aur unke beech connections
- Chapter 5: Backend — Spring Boot ka poora kaam, module by module
- Chapter 6: Security — JWT, password hashing, role-based access
- Chapter 7: Frontend — React ka poora kaam, page by page
- Chapter 8: Feature walkthroughs — step by step, jab user kuch karta hai toh peeche kya hota hai
- Chapter 9: Analytics — SQL aur Python wala data analysis part
- Chapter 10: Project kaise run karein apne computer par
- Chapter 11: Interview questions aur unke answers
- Chapter 12: Glossary — har technical word ka simple matlab

1. Project Overview — FleetX Kya Hai

1.1 Project ka Idea

FleetX ek **vehicle rental platform** hai — bilkul waisa hi jaise Zoomcar ya Ola-jaisi rental companies kaam karti hain, lekin chhote scale par, seekhne ke maksad se banaya gaya hai.

Simple shabdon mein: yeh ek website hai jahan koi bhi customer account banake, available cars/bikes dekh sakta hai, unhe kuch dinon ke liye book kar sakta hai, payment (simulate) kar sakta hai, aur baad mein review de sakta hai. Doosri taraf, ek **Admin** hota hai jo poore system ko manage karta hai — naye vehicles add karna, users manage karna, maintenance track karna, waghera.

1.2 Yeh Project Kyun Banaya Gaya

Iska maksad teen cheezein ek saath dikhana tha:

- **SDE / Full-Stack Development** — ek real-world jaisa web application banake dikhana ki tumhe frontend aur backend dono aata hai
- **Backend Security** — real applications mein security kaise implement hoti hai (login, encrypted passwords, permission-based access)
- **Data Analytics** — jo data application collect karti hai, usse SQL aur Python se analyze karke business insights nikalna

***Note:** Yeh ek 'intermediate-level' project hai — matlab beginner se thoda upar, lekin bahut complex bhi nahi. Ismein wahi cheezein hain jo real companies apne chhote internal tools mein use karti hain.*

1.3 Kaun Kya Kar Sakta Hai (Roles)

Role	Kya kar sakta hai
Customer (koi bhi)	Register/Login, vehicles dekhna aur search/filter karna, booking karna, payment karna, review dena
Admin	Sab kuch jo customer kar sakta hai, plus: vehicles add/edit/delete karna, sab users manage karna

2. Tech Stack — Har Technology Ka Matlab

Is project mein alag-alag kaam ke liye alag-alag tools/technologies use hui hain. Yahan har ek ko simple bhasha mein samjhaya gaya hai.

2.1 Frontend (jo user dekhta hai)

React

React ek JavaScript library hai jisse hum website ka **UI (User Interface)** banate hain — jo bhi screen par dikhta hai (buttons, forms, cards) woh React se bana hai. React ki sabse badi khoobi hai **components** — poori website ko chhote-chhote reusable pieces mein tod dena. Jaise ek 'VehicleCard' component ek hi baar likha, aur use har vehicle ke liye baar-baar use kiya.

React Router

Normal website mein alag-alag pages (Home, Login, Admin) hote hain. **React Router** yeh decide karta hai ki browser ke address bar mein jo URL hai, uske hisaab se kaunsa page dikhana hai — bina poori page reload kiye.

Axios

Axios ek tool hai jisse frontend, backend ko **request** bhejta hai — jaise 'mujhe saari vehicles ki list do' ya 'is user ko login karo'. Yeh JavaScript ke built-in 'fetch' jaisa hi hai, bas thoda aasan syntax deta hai.

2.2 Backend (jo peeche se kaam karta hai)

Java

Java ek programming language hai — backend ka saara logic isi mein likha gaya hai.

Spring Boot

Spring Boot, Java ke upar bana ek **framework** hai jo backend banane ka kaam bahut aasan bana deta hai. Bina Spring Boot ke, humein bahut saara repetitive/boilerplate code khud likhna padta — Spring Boot woh sab already deta hai (jaise web server chalu karna, database se connect karna).

Spring Security

Yeh Spring Boot ka hi ek hissa hai jo **security** handle karta hai — login check karna, kis user ko kaunse endpoint ka access hai, yeh sab isi se control hota hai.

2.3 Database

PostgreSQL

PostgreSQL (short mein 'Postgres') ek **database software** hai — yeh asli jagah hai jahan saara data (users, vehicles, bookings) permanently store hota hai, jaise ek bahut bada, organized Excel sheet.

2.4 Security Tools

JWT (JSON Web Token)

JWT ek tarika hai user ko 'login rehne' ka proof dene ka, bina baar-baar password bheje. Ise Chapter 6 mein detail se samjhaya gaya hai.

BCrypt

BCrypt ek algorithm hai jo password ko **encrypt (scramble)** kar deta hai save karne se pehle, taaki database dekhne wale ko bhi asli password pata na chale.

2.5 Analytics Tools

SQL

SQL (Structured Query Language) woh bhasha hai jisse hum database se sawal poochte hain — jaise 'sabse zyada revenue kis mahine mein aaya'.

Python + Pandas

Python ek programming language hai jo data analysis ke liye bahut popular hai. Pandas, Python ki ek library hai jo Excel-jaisi tables (data) ko handle karne mein madad karti hai.

Matplotlib

Yeh Python ki library hai jisse hum charts/graphs banate hain (bar chart, pie chart waghera).

2.6 Tools

- **Git** — version control tool, code ka har change track karta hai, taaki purane version par wapas ja saken
- **Postman** — backend ko directly test karne ka tool, bina frontend banaye
- **VS Code** — code likhne ka editor (IDE)
- **pgAdmin** — PostgreSQL database ko dekhne/manage karne ka visual tool

3. Poora Architecture — Sab Kuch Kaise Connect Hai

3.1 High-Level Picture

FleetX teen alag-alag hisson mein bata hai, jo ek doosre se baat karte hain:

```
React Frontend  ----->  Spring Boot Backend  ----->  PostgreSQL Database
(localhost:5173)          (localhost:8080)          (localhost:5432)

<----- JSON data wapas <-----
```

Frontend aur **Backend** do bilkul alag programs hain, alag-alag ports par chalte hain. Woh ek doosre se sirf **REST API** ke through baat karte hain — matlab frontend, backend ko ek 'address' (jaise /api/vehicles) par request bhejta hai, backend uska jawab JSON format mein deta hai.

3.2 Backend Ke Andar 4 Layers

Jab bhi koi request backend tak pahunchti hai (jaise 'mujhe vehicle id 3 ki details do'), woh 4 layers se guzarti hai — ek relay race jaisa:

Layer	Kaam
1. Controller	'Front desk' — request ko receive karta hai, decide karta hai kaunsa Service call karna hai. Khud koi logic nahi hai.
2. Service	'Decision maker' — asli business logic yahan hoti hai (jaise 'check karo vehicle available hai ya nahi')
3. Repository	'Database messenger' — sirf yehi layer database se seedha baat karti hai (data fetch/save karna)
4. Entity	'Blueprint' — Java class jo ek database table ko represent karti hai

Request ka safar: **Controller** → **Service** → **Repository** → **Database**, aur jawab wapas usi raste se ulta aata hai.

3.3 Frontend Ka Structure

Folder	Kya hai
components/	Chhote reusable pieces — Navbar, VehicleCard, LoginForm, ReviewForm, PaymentForm, Logo
pages/	Poore pages jo ek route (URL) par dikhte hain — HomePage, LoginPage, RegisterPage, MyBookings
context/	Global memory (UserContext) — 'kaun login hai' yeh info poore app mein available rakhta hai

4. Database Design — Saari Tables

FleetX ke database mein 6 tables hain. Har table ek 'Excel sheet' jaisi hai — rows (records) aur columns (fields) ke saath.

4.1 users table

Har registered person (customer ya admin) ka record.

Column	Matlab
id	Unique number, auto-generate hota hai
name	Poora naam
email	Login ke liye use hota hai, unique hona chahiye
password	Encrypted (BCrypt) form mein store hota hai, plain text kabhi nahi
phone	Contact number
role	CUSTOMER ya ADMIN — sirf yeh do values allowed hain
created_at	Account kab bana

4.2 vehicles table

Rent par milne wali har gaadi ka record.

Column	Matlab
id	Unique number
name	Jaise 'Honda City'
type	CAR ya BIKE
number_plate	Unique registration number
price_per_day	Rent price
seats	Seats ki sankhya
location	Kaunse city/branch mein hai
status	AVAILABLE / BOOKED / MAINTENANCE — teeno mein sirf ek ho sakta hai
image_url	Gaadi ki photo ka internet link
created_at	Kab add hui

Note: 'status' field bahut important hai — poora booking system isi ke upar depend karta hai. Jab booking hoti hai, status BOOKED ho jata hai; cancel hone par wapas AVAILABLE.

4.3 bookings table

Jab koi customer kisi vehicle ko book karta hai, ek row yahan banti hai. Yeh table users aur vehicles dono ko 'connect' karti hai.

Column	Matlab
id	Unique booking number
user_id	Kis user ne book kiya (users table ki id ki taraf pointer — 'foreign key')
vehicle_id	Kaunsi vehicle book hui (vehicles table ki id ki taraf pointer)
start_date, end_date	Rental ki date range
total_price	Automatically calculate hota hai: $\text{din} \times \text{price_per_day}$
status	ACTIVE / COMPLETED / CANCELLED
created_at	Booking kab hui

Note: 'Foreign key' ek simple concept hai — matlab yeh column doosri table ki ek row ki taraf 'ishara' karta hai, poora data dobara copy nahi karta.

4.4 payments table

Har booking ke liye simulate kiya gaya payment record.

Column	Matlab
id	Unique payment number
booking_id	Kaunsi booking ke liye (ek booking ka sirf ek hi payment ho sakta hai)
amount	Kitna paisa (booking ke total_price se liya jaata hai)
payment_method	CARD / UPI / CASH
status	SUCCESS ya FAILED (randomly simulate hota hai, 90% success)
paid_at	Payment ka time

4.5 maintenance_logs table

Jab admin kisi vehicle ko service/repair ke liye bhejta hai.

Column	Matlab
id	Unique record number
vehicle_id	Kaunsi vehicle
description	Kya kaam hua, jaise 'oil change'

cost	Maintenance ka kharcha
start_date, end_date	Kab shuru hua, kab khatam hua (end_date khaali ho toh matlab abhi bhi maintenance chal rahi hai)

4.6 reviews table

Booking khatam hone ke baad customer ki di hui rating/comment.

Column	Matlab
id	Unique review number
user_id	Kisne review diya
vehicle_id	Kaunsi vehicle ke baare mein
booking_id	Kaunsi booking se juda hai (ek booking ka sirf ek review ho sakta hai)
rating	1 se 5
comment	Likha hua review
created_at	Kab post hua

4.7 Tables Kaise Connect Hain (Relationships)

- Ek USER ke **many** BOOKINGS ho sakti hain
- Ek VEHICLE ki **many** BOOKINGS ho sakti hain
- Ek BOOKING ka **ek hi** PAYMENT ho sakta hai
- Ek VEHICLE ke **many** MAINTENANCE_LOGS ho sakte hain
- Ek BOOKING ka **ek hi** REVIEW ho sakta hai (double review allow nahi hai)

Note: Yeh 'ek se many' wala rishta database mein **Foreign Key constraint** se enforce hota hai. Isi wajah se agar kisi vehicle ki bookings hain, toh usse seedha delete nahi kar sakte — pehle uski bookings hatani padti hain, warna database error dega (yeh humne khud experience kiya tha project banate waqt).

5. Backend Explained — Module by Module

Har database table ke liye humne 4 Java files banayi (Entity, Repository, Service, Controller) — jaise Chapter 3 mein samjhaya. Ab har module ko detail se dekhte hain.

5.1 User Module

Entity (User.java)

Yeh Java class 'users' table ki blueprint hai. Ismein har column ke liye ek field hai (name, email, password, role, waghera). 'role' ek **enum** hai — matlab sirf CUSTOMER ya ADMIN, koi aur value allowed nahi.

Repository (UserRepository.java)

Yeh sirf 3 lines ki file hai, lekin isse hume automatically `save()`, `findAll()`, `findById()` jaise methods mil jaate hain, bina ek bhi SQL likhe. Ek special method bhi hai: `findByEmail()` — Spring Boot method ke naam se khud samajh leta hai ki kya query banani hai.

Service (UserService.java)

Yahan asli logic hai:

- **registerUser()** — check karta hai email pehle se exist toh nahi karta, password ko BCrypt se encrypt karta hai, fir save karta hai
- **login()** — email se user dhoondta hai, fir uska diya hua password aur database mein saved encrypted password ko compare karta hai
- **deleteUser(), updateRole()** — admin ke user-management ke liye

Controller (UserController.java)

Address (Endpoint)	Kaam
POST /api/users/register	Naya account banana
POST /api/users/login	Login karna, JWT token milta hai
GET /api/users	Sab users ki list (Admin only)
DELETE /api/users/{id}	User delete karna (Admin only)
PUT /api/users/{id}/role	Role badalna (Admin only)

5.2 Vehicle Module

Isi pattern par bana hai. Extra khaas baatein:

- **Status management** — jab vehicle add hoti hai, uska status automatically 'AVAILABLE' set hota hai
- **Filter methods** — `findByType()`, `findByStatus()`, `findByLocation()` — Spring Boot ke method-naming trick se automatically bante hain

- **Delete safety** — agar kisi vehicle ki bookings hain, delete karne par ek clear error milta hai ('Cannot delete this vehicle — it has existing bookings') na ki ek confusing crash

5.3 Booking Module — Sabse Interesting

Yeh module User aur Vehicle dono ko connect karta hai, aur real business logic yahan hai:

- Pehle check karta hai vehicle 'AVAILABLE' hai ya nahi — nahi toh reject
- `ChronoUnit.DAYS.between()` se automatically din count karta hai
- `total_price = din × price_per_day` — khud calculate hota hai, customer nahi bhar sakta
- Booking successful hone par vehicle ka status 'BOOKED' set kar deta hai
- Cancel hone par vehicle wapas 'AVAILABLE' ho jaati hai

5.4 Payment Module

Yeh **simulate** karta hai ki payment ho raha hai — koi asli bank involved nahi hai.

- Booking ka `total_price` hi payment ka amount ban jaata hai (customer khud amount nahi de sakta, fake kam price nahi bhar sakta)
- `new Random().nextInt(100) < 90` — 90% chance SUCCESS, 10% chance FAILED, jaise real payment gateway kabhi-kabhi fail hota hai
- Ek booking ka sirf ek hi payment ho sakta hai — dobara try karne par clear error message milta hai

5.5 Maintenance Module

- **startMaintenance()** — ek naya log banata hai, vehicle ka status 'MAINTENANCE' kar deta hai (taaki uss waqt koi book na kar sake)
- **completeMaintenance()** — log mein `end_date` daalta hai, vehicle ko wapas 'AVAILABLE' kar deta hai

5.6 Review Module

- Review dete waqt sirf `booking_id` diya jaata hai — user aur vehicle automatically usi booking se copy ho jaate hain (koi fake review nahi bana sakta kisi doosre ki booking par)
- Same booking par dobara review dena block hota hai

6. Security — JWT, Password Hashing, Roles

6.1 Password Kabhi Plain Text Mein Kyun Nahi Rakhte

Agar database hack ho jaye, aur password plain text mein ho, toh hacker seedha sabka password padh sakta hai. Isliye hum **BCrypt** use karte hain — ek 'one-way' encryption. Matlab: password ko scramble kiya ja sakta hai, lekin scrambled version se wapas asli password nikalna practically impossible hai.

```
Register karte waqt:
password = passwordEncoder.encode("1234")
// database mein save hota hai: $2a$10$efrLR... (ek lamba scrambled text)

Login karte waqt:
passwordEncoder.matches("1234", savedScrambledPassword)
// yeh 'true' ya 'false' return karta hai, kabhi bhi unscramble nahi karta
```

6.2 JWT (JSON Web Token) Kya Hai

Socho website ek concert hai. Jab tum ticket kharido (login karo), tumhe ek **wristband** (JWT token) milta hai. Ab jab bhi tum kisi gate (endpoint) se andar jaana chaho, tumhe bas apna wristband dikhana hai — baar-baar ticket (password) dikhane ki zaroorat nahi.

Token ke andar kya hota hai

Humara JWT token mein 3 cheezein hoti hain:

- **Email** — yeh batata hai token kis user ka hai
- **Role** — CUSTOMER ya ADMIN (isi se pata chalta hai kya permission hai)
- **Expiry time** — 24 ghante baad token khud expire ho jata hai

Yeh sab ek **secret key** se 'sign' (lock) kiya jata hai, taaki koi khud se fake token na bana sake.

Har Request Ke Saath Token Kaise Jaata Hai

Login hone ke baad, frontend token ko `localStorage` mein save kar leta hai. Har request ke saath yeh header automatically bhej diya jata hai:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ...
```

Backend mein ek special file **JwtAuthFilter.java** hai jo **har request** pe chalta hai. Yeh header padhta hai, token verify karta hai, aur agar sahi hai toh Spring Security ko batata hai 'yeh request is user ki hai, is role ke saath'.

6.3 Role-Based Access — Kaun Kya Kar Sakta Hai

Sirf login hona kaafi nahi — humne specific rules banaye hain ki kaun kaunsa endpoint use kar sakta hai:

Endpoint	Kisko Allowed Hai
GET /api/vehicles (dekhna)	Sabko — login zaroori nahi

POST/PUT/DELETE /api/vehicles	Sirf ADMIN
/api/maintenance/**	Sirf ADMIN
Users manage karna (list/delete/role)	Sirf ADMIN
Booking banana, apni history dekhna	Koi bhi logged-in user
Sab bookings/payments dekhna	Sirf ADMIN

Yeh sab **SecurityConfig.java** file mein define hai, aisi lines ke through:

```
.requestMatchers(HttpMethod.POST, "/api/vehicles").hasRole("ADMIN")
.requestMatchers(HttpMethod.GET, "/api/vehicles/**").permitAll()
.anyRequest().authenticated()
```

Agar koi CUSTOMER admin wala endpoint hit kare, backend seedha **403 Forbidden** error de deta hai — frontend tak pahunchne se pehle hi reject ho jata hai. Yeh humne khud test karke confirm kiya tha.

6.4 Yeh Sab Kyun Zaroori Hai (Interview Point)

Note: Bina JWT/role-check ke, koi bhi Postman se seedha 'DELETE /api/vehicles/1' bhej ke koi bhi vehicle delete kar sakta tha, bina login kiye. Real apps mein yahi sabse bada security risk hota hai. FleetX mein yeh gap identify karke fix kiya gaya — pehle simple 'localStorage mein user save karo' tarika tha, baad mein proper JWT + role check add kiya gaya.

7. Frontend Explained — Page by Page

7.1 React Ke Bunyadi Concepts

Component

Ek component bas ek JavaScript function hai jo HTML-jaisa content return karta hai. Jaise:

```
function VehicleCard({ vehicle }) {  
  return <div>{vehicle.name}</div>;  
}
```

JSX

Yeh HTML jaisa dikhta hai lekin asal mein JavaScript hai — isi se hum component ke andar UI likhte hain.

useState — Component Ki Memory

Jab bhi kisi cheez ko 'yaad' rakhna ho (jaise form mein kya type kiya), useState use karte hain:

```
const [email, setEmail] = useState("");  
// email = current value, setEmail = ise badalne ka function
```

useEffect — 'Jab Page Khule Tab Yeh Karo'

Jaise hi component screen par aata hai, useEffect ke andar likha code chalta hai — jaise backend se data mangwana:

```
useEffect(() => {  
  axios.get("http://localhost:8080/api/vehicles")  
    .then(res => setVehicles(res.data));  
}, []);
```

Props

Ek component doosre component ko data 'de' sakta hai — jaise `<VehicleCard vehicle={car} />`. Yahan 'vehicle' ek prop hai.

7.2 Context — Global Memory

Normal useState sirf usi component ke andar kaam karta hai. Lekin 'kaun login hai' jaisi info poore app mein chahiye (Navbar mein naam dikhane ke liye, Booking button mein user id ke liye). Isliye **UserContext** banaya — ek shared 'notice board' jo poora app padh/likh sakta hai.

```
const { currentUser, login, logout } = useUser();
```

Jab login hota hai, `login(user, token)` call hota hai — yeh Context ke andar user ka data save karta hai, `localStorage` mein bhi (taaki page refresh karne par bhi login yaad rahe), aur axios ko batata hai ki har request ke saath token bhejna hai.

7.3 React Router — Pages Ke Beech Navigation

Teen cheezein milke ek naya page 'wire' karti hain:

- **Page banao** (jaise `LoginPage.jsx`)

- **Route add karo** App.jsx mein — `<Route path="/login" element={<LoginPage />} />`
- **Navbar mein link do** — `<Link to="/login">Login</Link>`

7.4 Page-by-Page Breakdown

HomePage + VehicleList

Yahan hero banner aur vehicle listing hoti hai. Search box aur type-filter dropdown se list ko client-side filter kiya jata hai. Har vehicle **VehicleCard** component mein dikhti hai, jisme apna khud ka booking-form aur reviews hote hain.

RegisterPage / LoginPage

Simple forms — controlled inputs (value + onChange), submit hone par axios se backend ko POST request.

MyBookingsPage

Logged-in user ki saari bookings dikhata hai, har ek ke saath Cancel, Pay Now, aur Leave a Review buttons (toggle ho ke inline forms khulte hain).

AdminPage

Sabse bada page — stat tiles (revenue, active bookings), Add Vehicle form, All Vehicles list (delete button), Maintenance section, Manage Users section (role dropdown + delete), All Bookings list.

7.5 Axios Se Backend Baat Karna

Har jagah pattern same hai:

```
axios.post("http://localhost:8080/api/bookings/user/5/vehicle/1", {
  startDate: "2026-10-01",
  endDate: "2026-10-03"
})
.then(response => { /* success */ })
.catch(error => { /* kuch galat hua */ });
```

8. Feature Walkthroughs — Step by Step

8.1 Registration

- User form bharta hai (name, email, password, phone)
- Frontend axios se POST /api/users/register bhejta hai
- Backend check karta hai email pehle se hai kya
- Password BCrypt se encrypt hota hai
- Role automatically 'CUSTOMER' set hota hai
- Database mein naya row insert hota hai

8.2 Login

- Email/password backend ko jaata hai
- Backend password compare karta hai (encrypted se)
- Agar sahi hai, ek JWT token banta hai (email + role ke saath)
- Response mein user ka data aur token dono aate hain
- Frontend token ko localStorage mein save karta hai, aur Context update karta hai

8.3 Browse aur Book Karna

- HomePage load hote hi GET /api/vehicles call hoti hai
- User search/filter use karta hai (yeh sirf frontend mein hota hai, koi extra backend call nahi)
- 'Book Now' dabane par dates ke saath POST request jaati hai
- Backend check karta hai vehicle available hai, din count karta hai, price calculate karta hai
- Booking ban jaati hai, vehicle status 'BOOKED' ho jaata hai
- Frontend list ko refresh kar deta hai taaki status turant update dikhe

8.4 Payment Simulation

- 'Pay Now' dabane par method (CARD/UPI/CASH) ke saath request jaati hai
- Backend check karta hai is booking ka payment pehle se toh nahi hua
- Random simulate hota hai — 90% SUCCESS, 10% FAILED
- Result frontend par dikha diya jaata hai

8.5 Review Dena

- 'Leave a Review' se ek chhota form khulta hai — rating aur comment
- Submit hone par booking_id ke saath backend ko bheja jaata hai
- Backend usi booking se user aur vehicle nikal leta hai automatically

- VehicleCard par woh review turant dikhne lagta hai

8.6 Admin — Vehicle Maintenance

- Admin dropdown se vehicle chunta hai, description/cost/date bharta hai
- Maintenance start hote hi vehicle ka status 'MAINTENANCE' ho jaata hai (booking ke liye available nahi rehta)
- 'Mark Complete' dabane par end_date set hota hai, vehicle wapas 'AVAILABLE'

8.7 Admin — Users Manage Karna

- Sab users ki list dikhti hai naam, email ke saath
- Role dropdown se CUSTOMER/ADMIN badal sakte hain
- Delete button se user hata sakte hain (agar uski bookings hain toh clear error milega)

9. Analytics — SQL aur Python

9.1 SQL Queries

Yeh queries seedha database par chalti hain, real business questions ka jawab dene ke liye.

Total Revenue

```
SELECT SUM(amount) AS total_revenue
FROM payments
WHERE status = 'SUCCESS';
```

Sirf successful payments ka amount jodta hai.

Sabse Popular Vehicle

```
SELECT v.name, COUNT(b.id) AS total_bookings
FROM vehicles v
LEFT JOIN bookings b ON b.vehicle_id = v.id
GROUP BY v.id, v.name
ORDER BY total_bookings DESC;
```

LEFT JOIN use kiya taaki jin vehicles ki koi booking nahi hui, wo bhi 0 ke saath dikhein.

Cancellation Rate

```
SELECT COUNT(*) FILTER (WHERE status='CANCELLED') * 100.0 / COUNT(*)
AS cancellation_rate_pct
FROM bookings;
```

FILTER (WHERE ...) ek trick hai — isse hum ek hi query mein 'total count' aur 'condition wala count' dono nikal sakte hain.

Vehicle Utilization, Top Customers, Avg Booking Value

Yeh sab isi tarah ki queries hain — **GROUP BY**, **SUM**, **COUNT** combine karke real insights nikalna. Poori list `analytics/sql/queries.sql` file mein hai.

9.2 Python + Pandas

Yehi SQL queries Python mein bhi chalayi gayi hain, taaki data se **charts** bana sakein.

```
engine = create_engine("postgresql+psycopg2://...")
bookings = pd.read_sql("SELECT * FROM bookings", engine)

bookings_per_vehicle = bookings.groupby("vehicle_id").size()
bookings_per_vehicle.plot(kind="bar")
plt.savefig("output/bookings_per_vehicle.png")
```

- `pd.read_sql()` — SQL query chalake result ko ek table (DataFrame) mein le aata hai
- `.groupby().size()` — SQL ke **GROUP BY** + **COUNT** jaisa
- `plt.savefig()` — chart ko image file mein save karta hai

Yeh sab `analytics/python/analyze.py` file mein hai — jab bhi chahoge, dobara chalake fresh charts mil jayenge.

10. Project Kaise Chalavein

10.1 Zaroori Software

- Java (JDK 21+)
- Node.js (npm ke saath)
- PostgreSQL
- VS Code (ya koi bhi editor)

10.2 Pehli Baar Setup

- PostgreSQL mein 'fleetx' naam ka database banao
- backend/src/main/resources/application.properties file mein database ka username/password daalo
- Root folder mein: `npm install`
- Frontend folder mein: `cd frontend && npm install`

10.3 Har Baar Chalane Ke Liye

Root folder (FleetX) mein terminal khol ke bas yeh chalao:

```
npm run dev
```

Yeh command **backend aur frontend dono ek saath** start kar deta hai — backend localhost:8080 par, frontend localhost:5173 par.

Note: Yeh ek special setup hai jo humne 'concurrently' tool se banaya — normally 2 alag terminal khol ke 2 command chalane padte, ab sirf ek command se kaam ho jata hai.

10.4 Test Karne Ke Liye Login

Role	Email/Username	Password
Admin	admin@fleetx.com	admin123
Customer	ishdeep	1234

11. Interview Questions & Answers

Q: Apna project ek line mein batao.

A: FleetX ek full-stack vehicle rental platform hai jisme React frontend, Spring Boot backend, PostgreSQL database, JWT-based security, aur SQL/Python se analytics banayi hai.

Q: Frontend aur backend kaise baat karte hain?

A: Dono alag ports par chalte hain (frontend 5173, backend 8080). Frontend, Axios use karke backend ko REST API requests bhejta hai (GET/POST/PUT/DELETE), backend JSON format mein jawab deta hai. Do alag origins ke beech request allow karne ke liye CORS configure kiya hai.

Q: Backend mein layers kya hain aur kyun?

A: Controller, Service, Repository, Entity. Har layer ka ek hi kaam hai (separation of concerns) — isse code organize, testable aur maintainable rehta hai. Controller sirf request receive karta hai, Service mein logic hai, Repository database se baat karta hai.

Q: Authentication kaise kaam karta hai?

A: Login hone par backend ek JWT token generate karta hai jisme user ka email aur role hota hai, ek secret key se sign kiya jata hai. Frontend is token ko har request ke Authorization header mein bhejta hai. Backend ka JwtAuthFilter har request par token verify karta hai.

Q: Password kaise secure kiya hai?

A: BCrypt hashing algorithm se — yeh one-way encryption hai, matlab hash se wapas asli password nikalna practically impossible hai. Login ke waqt diya gaya password hash karke saved hash se compare kiya jata hai, kabhi bhi decrypt nahi kiya jata.

Q: Role-based access kaise implement kiya?

A: JWT token ke andar role bhi save hota hai. Spring Security ke SecurityConfig mein har endpoint ke liye rule likha hai — jaise 'POST /api/vehicles sirf ADMIN kar sakta hai'. Agar galat role try kare, 403 Forbidden milta hai, request controller tak pahunchti hi nahi.

Q: Database mein tables ke beech relationship kaise hai?

A: Foreign keys se — jaise bookings table mein user_id aur vehicle_id columns hain jo users aur vehicles table ki id ki taraf point karte hain. Isse data duplicate nahi hota, aur referential integrity maintain rehti hai (jaise ek vehicle delete nahi ho sakti agar uski bookings exist karti hain).

Q: Sabse mushkil bug kya tha aur kaise fix kiya?

A: Ek baar customer requests galti se 403 de rahi thi. Debug karne par pata chala yeh koi security logic ka bug nahi tha — Spring Boot DevTools ka hot-reload thoda stale ho gaya tha jab humne bahut saari security files ek saath edit ki. Poori tarah restart karne se fix hua. Isse yeh seekha ki har bug security-logic ka nahi hota — environment/tooling issues bhi same symptom de sakte hain, isliye systematically debug karna zaroori hai (logs padhna, ek-ek karke test karna).

Q: Vehicle booking ka price kaise calculate hota hai?

A: Backend khud calculate karta hai — start aur end date ke beech ke din count karke price_per_day se multiply karta hai. Frontend ya user isse manually nahi de sakta, isliye koi fake price nahi bhej sakta.

Q: Payment 'simulate' karne ka matlab kya hai?

A: Koi asli bank/payment gateway involved nahi hai. Hum sirf ek random number generate karte hain — 90% chance payment 'SUCCESS' hoga, 10% 'FAILED', jaise real duniya mein bhi kabhi-kabhi payment fail hota hai. Yeh sirf demonstrate karne ke liye hai ki payment flow kaisa dikhta hai.

Q: Analytics part mein kya kiya?

A: SQL queries likhi jo revenue, popular vehicles, cancellation rate, vehicle utilization, aur top customers nikaalti hain. Fir wahi analysis Python aur Pandas mein dobara kiya, aur Matplotlib se bar chart aur pie chart banaye jo visually data dikhate hain.

Q: Agar aur time milta toh kya improve karte?

A: Power BI se ek interactive dashboard banata (SQL/Python ke baad ka agla step), automated unit tests add karta, aur real file-upload se vehicle images add karne ka feature banata (abhi sirf image URL daalna padta hai).

12. Glossary — Har Term Ka Simple Matlab

API — Application Programming Interface — do programs ke baat karne ka tarika/address

REST API — Ek style of API jisme har cheez ek 'resource' hai (jaise /api/vehicles), aur GET/POST/PUT/DELETE se uspar kaam kiya jata hai

Endpoint — Ek specific address jahan request bheji ja sakti hai, jaise /api/users/login

JSON — Data ko likhne ka ek format — {"name": "Aman"} jaisa — jisse frontend/backend aasani se samajh sakein

Component (React) — Ek reusable piece of UI, ek JavaScript function ki tarah

State — Component ki 'yaadasht' — jo data badalta rehta hai (useState se manage hota hai)

Props — Data jo ek component doosre ko deta hai

Context (React) — Poore app mein shared data — bina har component ko manually pass kiye

Entity (Spring Boot) — Java class jo ek database table ko represent karti hai

Repository (Spring Boot) — Interface jo database ke saath seedha kaam karta hai

Service (Spring Boot) — Class jahan business logic likha jata hai

Controller (Spring Boot) — Class jo web requests receive karti hai

Dependency Injection — Spring khud zaroori objects bana ke de deta hai (@Autowired se), humein khud banana nahi padta

JWT — JSON Web Token — login ka encrypted 'proof', jisme user ki info hoti hai

BCrypt — Password ko encrypt (scramble) karne ka algorithm, one-way (wapas decode nahi ho sakta)

CORS — Cross-Origin Resource Sharing — browser ka security rule jo control karta hai kaunsi website kis backend ko request bhej sakti hai

Foreign Key — Database column jo doosri table ki row ki taraf 'point' karta hai

CRUD — Create, Read, Update, Delete — 4 basic operations jo har app mein hoti hain

GROUP BY (SQL) — Data ko groups mein baant ke, har group ka summary nikalna (jaise COUNT, SUM)

DataFrame (Pandas) — Python mein ek table jaisi data-structure, Excel sheet jaisi

Hot Reload / DevTools — Spring Boot ka feature jo code change hote hi server ko automatically restart kar deta hai, bina manually band-chalu kiye

localStorage — Browser ki apni storage, jahan data tab band karne ke baad bhi yaad rehta hai

Axios — JavaScript library jisse backend ko HTTP requests bheji jaati hain

Environment / .env — Configuration jaise database password, jo code se alag rakha jata hai

Yeh handbook FleetX project ki poori journey cover karti hai — planning se leke deployment-ready state tak. Isse padhne ke baad tumhe apna khud ka project confidently explain karna aa jaana chahiye.